Building a multi-agent workflow in Haskell

A demonstration of the benefits of strong type systems for agentic workflows (and a hurrah to pure functional programming).

Introduction to Haskell

I was introduced to Haskell by a tutor leading a CS course 'Algorithms and Data structures'. Despite the fact that 9 months of course material was all written using python, the tutor opened the first lecture by dismissing it as rubbish and proceeded to teach the entire course using Haskell. After a very confusing first couple of months (no iteration, no variables and a compiler stricter than a dickensian headmaster), supplemented by a few hundred quid on World of Books, the penny dropped. Haskell is cool! limited? definitely, widely used? no, of career benefit? no, but who cares, its not python and that makes it cool.

Haskell is a purely functional programming language. The basic adage is, a Haskell programme describes what result you want, rather than specifying how to compute it step by step. It is statically typed with powerful type inference and absolute immutability - at runtime, you get the same output every time. The initial learning challenge is getting over the loss of iterative loops (counters are mutable so are illegal in pure functions). This drives a completely different way of thinking to structure functions based on recursion. The result being, arguably, actually quite elegant!

```
insertionSort :: (Ord a) => [a] -> [a]
insertionSort [] = []
insertionSort (x:xs) = insert x
    (insertionSort xs)
    where
        insert :: (Ord a) => a -> [a] -> [a]
        insert x [] = [x]
        insert x (y:ys)
            | x <= y    = x : y : ys
            | otherwise = y : insert x ys
```

This example of insertionSort gives us a constrained polymorphic function that will work for any type 'a' which is can be compared and ordered (int, integer, float, char, string, maybe, custom adts etc). Note the recursion (and nested recursion) and the two base cases which end the recursive loop. The notation (x:xs) is simply the head (x) and the rest (tail xs) of the list. Now tell me thats not neat!

Although Python dominates the AI landscape, Haskell's strengths align well with the challenges of agent orchestration. In a multi-agent system, messages flow between components with specific structures. Haskell's type system ensures that an A2A message always contains the required fields - you cannot accidentally pass a malformed message and cause a mess.

The idea for the app was to create 3 agents and 1 facilitator using different llms and trigger a debate between them (yes this has been done a million times before I am sure, but perhaps not with haskillian agents!). I tried to mimic the A2A protocol for messages, which were passed between agents as json files and converted to Ollama's own prompt format.

```
data A2AMessage = A2AMessage
{ msgId :: Text -- Unique message identifier
, sender :: Text -- Name of the sending agent
, recipient :: Text -- Target agent or "all"
, content :: Text -- The actual message content
, timestamp :: UTCTime -- When the message was created
, msgType :: MessageType -- Type of message
} deriving (Show, Eq, Generic)
```

Handling Impure Operations

But wait! LLM calls are impure! They involve completely non-deterministic responses, so we need to break out of the pure world and talk to the real world. Haskell does this using Monads (as apparently do a few other languages, they are just called something less explicit). Have a look at the function used to call Ollama:

```
callOllama :: Text -> Text ->
    [A2AMessage] -> IO (Either String Text)
callOllama modelName agentCardText
history = do
    manager <- newManager
tlsManagerSettings

    let ollamaMsgs = toOllamaMsgs
agentCardText history
        let reqBody = OllamaRequest
                { model = modelName
                , ollamaMsgs =
ollamaMsgs
                , stream = False
                }
```

• IO is the monad here, it describes the I/O actions that the runtime executes. The 'do' notation lets you sequence these actions. I asked if this was effectively imperative code, but was swiftly assured that it is not (although I am still trying to get my head round why not!)

• The manager utility is from the haskell lib Network.HTTP.Client which is pretty much the same as the requests lib in Python.

• toOllamaMsgs calls a function that converts the A2A message to Ollama's format.

• agentCardText is the simplified A2A Agent Card that I use (agents currently don't have any tools... the reality is that python beats haskell hands down when it comes to the simplicity of creating MCP).

• history is the cumulative history of the debate, so that the agents can make arguments with context.

Managing State in the Agent Cafe

The Agent Cafe as the programme is called, simulates 5 rounds of debate between 3 personalities (A philosopher, an economist and the CEO of a large enterprise). The orchestration requires managing state, in this context, keeping track of the rounds, who's turn it is, and the history of the conversation. This is easy in Python, and actually not too complex in Haskell. Haskell provides a library mtl (Monad Transformer Library) which adds state threading capability to monads.

```
agentTurn :: Agent -> MessageType ->
    Debate ()
agentTurn agent mType = do
    state <- get
```

StateT operates as you would expect; it has functions get, put, modify and liftIO, the last of which allows us to run I/O operations within the Debate monad.

```
get :: Debate DebateState
-- Returns the current state

put :: DebateState -> Debate ()
-- Replaces the entire state

modify :: (DebateState -> DebateState) ->
    Debate ()
-- Applies a function to transform the
state
```

Here's how each agent takes their turn in the debate using the above

```
agentTurn :: Agent -> MessageType ->
    Debate ()
agentTurn agent mType = do
    state <- get

    -- Generate response from agent
    liftIO $ do
        TIO.putStrLn $ "... " <>
agentName agent <> " is thinking ..."

    msg <- liftIO $ agentRespond agent
mType state

    -- Update state with new message
    modify (\s -> s { messages =
messages s ++ [msg] })

    -- Print the message
    liftIO $ printMessage msg
```

Calling the main method

```
main :: IO ()
main = do
    let initialState = emptyDebateState
topicText defaultRounds
        (_, finalState) <- runStateT
runDebate initialState
            saveTranscript finalState

defaultTopic :: Text
defaultTopic = "How will AI impact the
world over the next 10 years?"

defaultRounds :: Int
defaultRounds = 5

saveTranscript :: DebateState -> IO ()
saveTranscript state = do
    let filename = "debate_transcript.json"
```

This creates a debate containing the topic, an empty message list and a round counter set to zero. (_, finalState) <- runStateT runDebate initialState saveTranscript finalState is the key line, runStateT takes the initial state and executes the entire debate. It returns a tuple (_, finalState). The _ is a discarded value, it is a placeholder for runDebate :: Debate (), which returns nothing, so we ignore it. finalState is the accumulated state after all of our monad operations have completed, namely the debate transcript.

The Agents & LLMs

Part of the idea of the project was to get 3 different open-weight models running on the spark, which proved... remarkably easy. Ollama really does make running models straightforward. The config was as follows:

Philosopher - Qwen 2.5 32B
CEO - Llama 3.1 70B
Economist - Gemma 2 27B

Inference was quick (I didn't time exactly) and there there were no resource issues, so not really a test of the spark, more a utilisation of its memory for a short period of time.

So.... What Happened?

Initially, something very weird but equally fascinating. My original design had a 'Barista' agent who's job it was to introduce the debate topic, which was prompted as

> ***How will AI impact the world over the next 10 years?***

Everything worked, except the barista hallucinated and introduced a debate in Arabic about ancient Rome "From the platforms and public facilities of the forum".

I quickly decided to disempower him as a host and pass more instructive prompt directly to the 3 debate participants, essentially hardcoding it.

The debate was actually interesting, so I will publish it as its own article. Here is the opening gambit from the Philosopher:

> *Philosophically, we must question whether the integration of AI aligns with Aristotelian ideals of eudaimonia—human flourishing—or if it leads to a form of instrumentalism where individuals are seen merely as data points for optimization. The Kantian notion of treating individuals as ends in themselves, rather than means to an end, is particularly relevant here. If AI systems make decisions that override human autonomy or if they perpetuate biases embedded within their training data, we risk undermining the dignity and rights of individuals.*

I will publish the programme, it is remarkably short and simple, but lets face it, its Haskell so wont get used!